Some Misconceptions about Software in the Copyright Literature

Joshua Bloch†
Institute for Software Research
Carnegie Mellon University
Pittsburgh PA USA
jbloch@andrew.cmu.edu

Pamela Samuelson‡
Berkeley Law School
University of California, Berkeley
Berkeley CA USA
psamuelson@berkeley.edu

ABSTRACT

The technical complexity and functionality of computer programs have made it difficult for courts to apply conventional copyright concepts, such as the idea/expression distinction, in the software copyright case law. This has created fertile ground for significant misconceptions. In this paper, we identify fourteen such misconceptions that arose during the lengthy course of the *Google v. Oracle* litigation. Most of these misconceptions concern application programming interfaces (APIs). We explain why these misconceptions were strategically significant in Oracle's lawsuit, rebut them, and urge lawyers and computer scientists involved in software copyright litigation to adopt and insist on the use of terminology that is technically sound and unlikely to perpetuate these misconceptions.

CCS CONCEPTS

• Social and professional topics ~ Computing / technology policy ~ Intellectual property ~ Copyrights • Software and its engineering ~ Software libraries and repositories

KEYWORDS

API, copyright, software interface, intellectual property, IP law, compatibility, interoperability, Oracle v. Google, Google v. Oracle

ACM Reference format:
Joshua Bloch & Pamela Samuelson. 2022. Some Misconceptions about Software in the Copyright Literature. In *Proceedings of the 2022 ACM Symposium on Computer Science and Law (CSLAW'22), November 1–2, 2022, Washington, DC, USA. ACM, New York, NY, USA, 11 pages.* https://doi.org/10.1145/3511265.3550449

1. INTRODUCTION

In 1980, Congress amended the Copyright Act of 1976 to extend that law's protection to computer programs.[21] Programs satisfy the Act's broad definition of the term "literary work," a category of copyrightable subject matter defined as a work "expressed in words, numbers, or verbal or numerical symbols."[19:101]

Early case law established that all computer programs, not just application programs that produce visual displays, are protectable by copyright law.[22] These cases held that exact copying of original program code was copyright infringement. Since the mid-1980s, however, courts have found it difficult to decide which aspects of programs apart from code are within the scope of the protection that copyright law provides to software.

Congress tried to ensure that the scope of copyright in programs would not be construed too broadly by providing in Section 102(b) of the Copyright Act of 1976 that "[i]n no case does copyright protection extend to any idea, process, procedure, system, method of operation, concept, principle, or discovery regardless of the form in which it is described, explained, illustrated, or embodied in such work."[19:102(b)] Courts often refer to this as the *idea/expression distinction:* copyright protects expressive works, not underlying ideas[1]. But courts have often been uncertain about how to apply these terms of exclusion in software copyright cases.

One of the longstanding controversies has been over whether interfaces that enable compatibility are systems or methods of operation that § 102(b) excludes from copyright's scope, or are unprotectable under other copyright doctrines such as fair use.[15] Although most courts have rejected interface copyright claims, the case law and commentary on this issue has long been somewhat mixed. This controversy came to a head in a decade-long lawsuit brought by Oracle America, Inc. against Google Inc. for its reuse of parts of the Java Application Programming Interface (API).

For those unfamiliar with the term, an API is a set of subroutine definitions or communication protocols that provides access to a set of functionalities on the basis of their inputs and outputs, independent of their implementation. In other words, APIs are the

[1] Broadly constituted to include all of the excludable categories identified in § 102(b).

This work is licensed under a Creative Commons Attribution International 4.0 License.

CSLAW'22, November 1–2, 2022, Washington, DC, USA.
© 2022 Copyright is held by the owner/author(s).
ACM ISBN 978-1-4503-9234-1/22/11. https://doi.org/10.1145/3511265.3550449

†Joshua Bloch worked for both Google and Sun Microsystems (which was renamed Oracle America after its acquisition). He wrote many of the libraries at issue, and testified as a fact witness in both of the Oracle v. Google trials. He was lead computer scientist in five amicus curiae briefs to appellate courts in that case.[34,40,43,45,48] He also wrote books about the Java APIs[3,5,9] and taught a course entirely devoted to API development at Carnegie Mellon University.
‡Pamela Samuelson is a Richard M. Sherman Distinguished Professor of Law at Berkeley Law School and a Contributing Editor of Communications of the ACM. She was the lead author of five amicus curiae briefs in the Oracle v. Google case on behalf of intellectual property scholars in support of Google's defenses.[30,35,44,46,49] All of the briefs filed with the U.S. Supreme Court in that case can be found online.[55]

vernaculars that let the parts of a program talk to one another. A simple API for a sorting function is shown in Section 2.1.

In 1996, Sun Microsystems introduced the Java platform. The story of the *Google v. Oracle* litigation began in 2010, when Oracle acquired Sun, including all of its intellectual property rights, and almost immediately sued Google in federal court over its use of Java[2] in Android. The suit dragged on for eleven years before the Supreme Court ruled in Google's favor in April, 2021.[51] For most of the lawsuit's duration, the key issue was whether Android's reimplementation of 37 packages in the Java API constituted copyright infringement.

Google's principal defense was that the Java API declarations it used in Android were unprotectable by copyright law (a defense that both authors supported). Its second defense was that, even if APIs were copyrightable, Google's reimplementation was fair use.

In 2012, the first *Oracle v. Google* trial[28] was held in the U.S. District Court for the Northern District of California, and presided over by Judge William Alsup. Oracle's complaint involved both patent and copyright claims. The jury found that Google had not infringed any Oracle patents, and Judge Alsup ruled the APIs were not entitled to copyright protection as a matter of law. In 2013, Oracle appealed only the copyright decision, but because the case initially contained patent claims, the Court of Appeals for the Federal Circuit (CAFC) had jurisdiction over the appeal.

In 2014, the CAFC reversed Judge Alsup's ruling that APIs were uncopyrightable, and sent the fair use issue to the lower court jury.[32] Google asked the Supreme Court to hear its appeal of the CAFC ruling, but the Court declined this request.[37] In 2016, the jury in Judge Alsup's court rendered a verdict in favor of Google's fair use defense,[38] and again Oracle appealed. In 2018, the CAFC ruled that no reasonable jury could have found Google's use of the declarations was fair—which is perhaps surprising given that the CAFC had directed Judge Alsup to send the question to the jury in the first place.[42]

In 2019, Google asked the Supreme Court to review the CAFC's rulings on both copyrightability and fair use. This time, the Court granted Google's petition. In 2020, the Supreme Court heard oral arguments in *Google v. Oracle.*[3] In 2021, Justice Stephen Breyer wrote the majority decision upholding Google's fair use defense as a matter of law.[51] The Court did *not*, however, rule on whether APIs were copyrightable, but assumed "for argument's sake" that they were.[51 at 1187]

We commend Judge Alsup and Justice Breyer for their successful efforts to apply copyright law to software using accurate computer science concepts.[4] This was due in part to Judge Alsup's prior programming experience and his willingness to learn some Java in preparation for the trial.[12] Justice Breyer, who, in 1970 wrote his tenure article at Harvard about computer program copyright issues,[6] described the Supreme Court's *Google* decision as "one of the most important decisions I wrote this last year."[54] We agree wholeheartedly.

Throughout this litigation, in which both authors were active, we encountered misconceptions about software, APIs, and compatibility. We suspect that Oracle's lawyers introduced most of these misconceptions for strategic reasons, although some predate the lawsuit. The CAFC embraced many of these misconceptions in its rulings.[32,38]

This article identifies fourteen misconceptions concerning software and APIs that were manifest in the *Oracle v. Google* litigation, and offers rebuttals. Because the Supreme Court reversed the CAFC ruling on Google's fair use defense but not on its copyrightability defense, these misconceptions are likely to persist in future cases. This is especially true given that software copyright plaintiffs can ensure that any appeal will go to the CAFC by adding a patent claim to their complaint, even if the claim is without merit.

2. REBUTTED MISCONCEPTIONS

2.1 The API declarations at issue in *Oracle v. Google* are not computer programs; they are interface specifications.

Oracle persuaded the CAFC that the Java API declarations that Google reimplemented in Android were computer programs.[32] Justice Clarence Thomas's dissent in the *Google* case likewise accepted the conception of the declarations as programs.[51 at 1211 (Thomas, dissenting)] Judge Alsup, by contrast, understood the distinction between interfaces and implementations, as did the Supreme Court majority, which spoke of the Java declarations as "inextricably bound" with method calls used by Java programmers (in which Oracle claims no rights), and with implementing code (which is copyrightable, but which Google didn't copy).[51 at 1201] The Court's decision correctly characterized the declarations as an interface connecting method calls with implementing code.

The distinction between interfaces and implementations is both critical and subtle. From the standpoint of copyright law, computer programs are characterized as literary works, but they are also functional entities. Copyright protects a program's literary *expression*, but not the resulting *function*. A software library's functional specifications are precisely its API, which describes its functionality in terms of its inputs and outputs. The *only* way to write a new library that provides the same functionality as an existing one is to reimplement its API.

In essence, a software library is a machine, and its API is the method by which one operates that machine. This insight was noted by Judge Alsup in his ruling.[28] It is significant because Section 102(b) of the Copyright Act of 1976 specifically exempts methods of operation from copyright protection.[19] While the CAFC overturned Judge Alsup's ruling that APIs are uncopyrightable

[2] When we use the name Java without further qualification, we are referring to the platform released under the name Java in 1996, renamed "Java 2, Standard Edition" (J2SE) in 1998, and then renamed "Java Standard Edition" (Java SE) in 2004.

[3] In the Supreme Court, the case name was changed from *Oracle v. Google* to *Google v. Oracle*, because the Supreme Court names its cases by the party appealing a lower court ruling (the *petitioner*).

[4] Examples can be found in the Statement of Findings in Judge Alsup's decision in the first *Oracle v. Google* trial, which provides a fine tutorial on the structure of the Java platform;[28 at 977], and in Appendix B of the Breyer opinion, which provides a diagram and explanation of the Java API declarations in relation to method calls and implementing code.[52 at 38]

methods of operation, and the Supreme Court did not rule on the issue, we believe the ruling is correct from a technical perspective.

Another subtle point that deserves mention is that declarations are not complete API specifications. They communicate only the names and types of API elements, but not their semantics. For example, consider this Java method declaration:

```
public class Arrays {
  public static void sort(int[] a);
}
```

This declaration identifies a method (also known as a function or procedure) named `Arrays.sort` that takes a single input argument (also known as a parameter) which is an array of 32-bit signed integers. The declaration does not say anything about what the method actually does (its semantics). The semantics are typically specified in a natural language (such as English), a formal mathematical language, or some combination of the two. In the case of `Arrays.sort`, the semantic part of the API specification is simply the phrase "Sorts the specified array into ascending numerical order." The information conveyed by the declaration and the semantics together comprise the complete API (specification).

Critically, the API (declaration plus semantics) does not include the instructions that tell the computer *how* to sort the array into ascending numerical order. It is the *implementation code* (which must conform to the API specification) that accomplishes that task. It is this implementation code that falls squarely within copyright law's definition of a computer program as "a set of statements or instructions to be used directly or indirectly in a computer in order to bring about a certain result."[5][19:101]

Neither the entire API nor the declarations alone fall within the legal definition of a computer program. While declarations *do* influence the executable code, by telling the computer what sequence of argument types to expect and what argument type (if any) to return, they do *not* "bring about [the method's] certain result."[19:101]

2.2 "Declaring code" is a contrived and misleading substitute for the commonly accepted term of art, "declaration."

Oracle's lawyers likely coined the term "declaring code" when preparing the company's appeal of Judge Alsup's ruling that Google's reimplementation of the declarations did not infringe Oracle's copyright in Java.

At the trial court level, the *Oracle v. Google* case was litigated as a "nonliteral infringement" case. Judge Alsup's copyrightability decision noted that "all agreed that Google had not literally copied [Oracle's] software, but had instead come up with its own implementations." His decision emphasized that "Oracle's central claim was that Google had replicated the structure, sequence, and organization (SSO) of the overall code for the thirty-seven API packages."[28 at 975]

When appealing Judge Alsup's uncopyrightability ruling to the CAFC, Oracle adopted the terms "declaring code" and "declaring source code" in order to convert the nonliteral infringement case into a literal infringement case, making Google's reimplementation of the declarations look more like piracy to a technically unsophisticated audience.

Copyright lawyers use the term "literal infringement" to signify when someone has made an exact copy of another's work (or some substantial part of that work). Exact copying may sometimes be fair use, as when someone reverse engineers a program to discern its interfaces. It is, however, much easier to convince a judge or jury that someone is an infringer when a lawsuit involves a literal infringement claim.

The term "nonliteral infringement" signifies that a defendant is charged with copying a more abstract element of a copyright owner's work. In a case about whether a novel infringes the copyright in a play, for example, a defendant's appropriation of detailed sequences of events from important scenes might well constitute nonliteral infringement.

In software copyright cases, it is often quite difficult to prove nonliteral infringement because courts have recognized that many nonliteral elements of programs, such as algorithms, are not within the scope of protection that copyright law provides. Several cases have regarded program interfaces that enable compatibility as nonliteral elements, and held that reuse of those interfaces to achieve compatibility does not infringe copyright.[15:11–25,24,25,27]

By adopting the term "declaring code" as part of Oracle's appeal to the CAFC, Oracle's lawyer managed to persuade the CAFC judges that Google had literally infringed Oracle's copyright by copying several thousand lines of "declaring code" and had also nonliterally infringed by copying "the elaborately organized taxonomy of all the names of methods, classes, interfaces, and packages."[32 at 1351]

Google's Supreme Court counsel informed us that he decided to accept the term "declaring code" when briefing the Supreme Court during Google's appeal even though he knew that "declarations" was the correct term.[10] He thought that the case was already very complicated and fighting over the "declaring code" term would only make it more complicated. It is hence unsurprising that the Supreme Court's *Google* decision referred to the Java API elements at issue as "declaring code." Fortunately, the Court was not misled into characterizing the *Google* case as a literal infringement case. The Court properly identified the declarations as an interface between method calls and implementing code.

2.3 Google did not invent the term "software interface" during the *Oracle v. Google* litigation.

Oracle's merits brief to the Supreme Court had the temerity to claim that "Google carve[d] out from copyright protection a category of

[5] The wording of this definition clarifies that copyright protection extends to both source and object code: the source code brings about its "certain result" indirectly via its object code.[19:101]

computer code that it vaguely calls 'interfaces.'"[50:2] Not only does this conflate interface specifications with computer programs, but it suggests that software interfaces are some sort of vague notion that Google contrived for the purpose of this litigation. Nothing could be further from the truth.

The term *application program[ming] interface* has been part of our lexicon at least since 1968,[8] and the term *software interface* has been in the lexicon at least since 1965[16] and probably earlier. The Google Books Ngram Viewer[56] (whose corpus extends from 1800 to 2019) shows the term *software interface* rising to prominence in the 1960s, and *application programming interface* rising to prominence in the 1980s. By contrast, the corpus contains virtually *no* uses of the term *declaring code* (Figure 1).

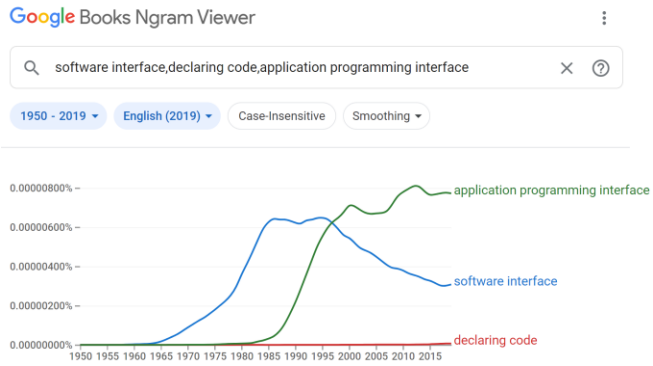

Figure 1: Usage of terms software interface, application programming interface, and declaring code[57]

Searching for the phrase "declaring code" in Google's index of scholarly works shows that virtually *every* occurrence of the phrase occurs in a legal paper discussing the *Oracle v. Google* case.[58] Simply put, software interfaces, also known as APIs, are a longstanding, fundamental concept in computer science, whereas "declaring code" is a term likely invented by Oracle in the hope of improving its chances of winning its legal case against Google.

2.4 Google almost certainly did not copy the declarations from the Java library source code.[6]

As explained in Section 2.2, Oracle tried to turn its case into a literal copying case by claiming that Google copied its declarations. It would have been *much* more difficult for Google to copy the declarations than to generate them using `javap`, a tool that Sun (and later, Oracle) provided expressly for the purpose of generating declarations from class files (which are Java's form of object code). Copying the declarations from the source code would also have been much more error-prone than using `javap`. Realistically, there is *no* reason that an engineer would have chosen to copy declarations from the source code rather than using `javap`.

This raises the question of why Oracle would provide this tool if they didn't want developers to use it. The answer is that they *did* want developers to use it, because reimplementing existing APIs is part of the daily life of the typical Java programmer. For example, if you want to implement an interface type, to write a concrete implementation of an abstract class, or to change the behavior of a concrete class, you must write new implementations of the methods it declares.

In the old days, before Integrated Development Environments (IDEs) were commonly available, programmers would use `javap` to produce a set of declarations to supplement with new implementations. Nowadays, IDEs further automate the process, but either way, a tool extracts the relevant information from the platform's object code and automatically generates source code for compatible declarations.

If Google's actions in reimplementing the Java APIs had ultimately been found to violate the copyright in those APIs, then essentially *all* Java programmers would be guilty of the same offense, albeit on a smaller scale.

It is also worth noting that Google could have reimplemented the Java APIs from a natural language description of each API element, such as the English description of `Arrays.sort` immediately following its declaration in Section 2.1. The takeaway here is that API reimplementation is not prima facie evidence that API declarations have been copied literally. Of course, this is immaterial if API declarations are not protectable by copyright, or if copying them in order to reimplement APIs constitutes fair use (as the court ruled in *Google v. Oracle*).

2.5 Recognizing the legality of reimplementing APIs does not risk eliminating copyright protection for software altogether.

Google persuaded Judge Alsup that the Java declarations it used in Android were constituent elements of an unprotectable method or system that copyright law excludes from protection under 17 U.S.C. § 102(b). As noted in the introduction, that provision states that "[i]n no case does copyright protection for an original work of authorship extend to any idea, procedure, process, system, method of operation, concept, principle, or discovery, regardless of the form in which it is described, explained, illustrated, or embodied in such work."[19:102(b)]

Judge Alsup's ruling was consistent with two previous decisions in which the Ninth Circuit Court of Appeals held that program interfaces needed for achieving compatibility are unprotectable under § 102(b).[24,27] (The CAFC was supposed to follow Ninth Circuit case law in *Oracle v. Google* because the trial court was located in that circuit.) Those decisions recognized a clean distinction between interfaces and implementations.

However, Oracle argued to the appellate courts that there was no bright line separating "declaring code" from other computer code, so allowing others to reimplement the Java declarations, either by recognizing their uncopyrightability or by deciding a reimplementation was fair use, would, in effect, eliminate copyright

[6] In *Oracle v. Google*, Google stipulated to copying the Java API declarations, i.e., they conceded it for the purpose of the litigation, but that doesn't make it true.

protection for computer code altogether because all computer programs are functional.[31:16]

Unfortunately, the CAFC's 2014 *Oracle* decision and Justice Thomas's dissenting opinion, joined by Justice Samuel Alito, found Oracle's argument persuasive.[32,51 at 1211 (Thomas, dissenting)] These judges did not understand that, far from being difficult to separate API declarations from executable code, it is so easy that it can be trivially automated. As mentioned in the previous section, this is precisely what the `javap` tool does by default when given a class file.

2.6 Structure, sequence, and organization (SSO) is not a meaningful term in the software field.

In the software field, "structure, sequence, and organization" is a technically incoherent term. Software professionals don't use the phrase, let alone the acronym. SSO means something entirely different to them (*single sign-on).* Software professionals would not understand the term as a meaningful description of what lawyers call nonliteral elements of programs.

This is not to deny that programs embody structural elements, that they contain sequences of elements such as statements to be executed, or that they may be organized in various ways. But stringing those three words together as a phrase does not make them into a coherent conceptual framework for understanding computer programs or their component elements.

SSO was introduced into the legal literature by software copyright lawyers who wanted to persuade courts that computer programs should be accorded a broad scope of copyright protection. The Third Circuit Court of Appeals endorsed this phrase in *Whelan Associates, Inc. v. Jaslow Dental Laboratories, Inc.*, 797 F.2d 1222 (3d Cir. 1986).[23] That case arose because after a falling out with Whelan, Jaslow developed a dental laboratory business program to compete with the one he had hired Whelan to develop. Whelan persuaded both a trial judge and an appellate court that Jaslow infringed by copying the overall structure of her program, certain file structures, and sequences of operations for some modules.

Because computer programs fall within copyright's broad statutory definition of "literary works," the *Whelan* court reasoned that if the SSO of novels was protectable by copyright law, so should be the SSO of programs. For the first six years after the *Whelan* decision, many software copyright plaintiffs successfully raised SSO claims against competitors.

In 1992, the Second Circuit Court of Appeals—the most influential court in copyright jurisprudence—heavily criticized the *Whelan* decision and its endorsement of SSO claims in *Computer Associates International, Inc. v. Altai, Inc.*, 982 F.2d 693 (2d Cir. 1992).[25] That court ruled that courts must not find infringement based on similarities in the services that programs perform or parameter lists that facilitate compatibility with other programs.

The Second Circuit in *Altai* recognized that SSO was not a meaningful term in the computing field.[25] While that court agreed that some nonliteral elements of programs might be protectable by copyright law, it regarded SSO as too vague and likely to encompass too many nonliteral elements of programs that copyright should not protect. The Second Circuit directed courts to filter out many types of unprotectable elements of computer programs before trying to assess whether someone infringed another's original program.

Until the CAFC's 2014 *Oracle* decision, SSO had largely fallen out of use in software copyright case law. In its place, courts across the country adopted *Altai's* highly dissective approach to assessing software copyright claims, known as the *Abstraction-Filtration-Comparison test.*[25] Judge Alsup's 2012 *Oracle* decision, like *Altai,* abjured the SSO concept.[28]

Oracle, however, insisted that Google had infringed by copying the SSO of Java's API declarations, and persuaded the CAFC to adopt this terminology. That court upheld Oracle's claim of infringement based on Google's copying of the Java API SSO, as well as its copying of the declarations.[32]

Although the Supreme Court's *Google* decision mentioned that Oracle claimed infringement of this SSO, the Court said nothing about the protectability of program SSO. Yet, it did not vacate the CAFC's 2014 decision. This will likely mean that SSO claims will continue to be made in software copyright cases slated to go to the CAFC on appeal. As previously suggested, plaintiffs will likely make a habit of adding patent claims to software copyright cases to drive the appeal into the CAFC.

2.7 It is inappropriate to equate API declarations to structural elements of novels and the like.

Oracle's merits brief to the Supreme Court contained numerous grossly exaggerated analogies suggesting that Google's reuse of the Java API declarations was akin to stealing the most valuable elements of fictional or other conventional copyrighted works: "By Google's logic," Oracle argued, "a plagiarist could define J.K. Rowling's idea as 'a story about Harry Potter, Ron Weasley, and Hermione Granger who attend Hogwarts' and steal the characters and their backstories" and could then "reproduce verbatim the 11,300 most memorable sentences or scenes because they were 'necessary' to allow fans to use their existing knowledge."[50:34]

Oracle argued that Google should not be able to copy declarations "based on a desire to appeal to what the audience knows about the original," likening this to "a conductor who wants to perform songs his orchestra already knows or an author who wants to write a *Seinfeld* trivia book based on familiar dialogue."[50:52–53]

Fortunately, the Supreme Court disregarded these inapt analogies and recognized that the functional nature of programs and of API declarations required a very different framing of the copyright interests at stake. The Court was quite clear that the bulk of the value residing in the declarations was due to programmers' investments in learning and using them, not in what Sun or Oracle had done when it initially created them.

2.8 Just because writing an API requires creativity does not mean it is eligible for copyright protection.

Throughout the litigation, Oracle made a big deal out of the fact that writing a good API requires creativity. In support of this contention,

they pointed to a lecture by one of the authors (Bloch) that went into great depth about the craftsmanship required to come up with a good API.[2] Thus, claimed Oracle, APIs *must* be eligible for copyright protection. In fact, creativity is a necessary condition for copyrightability, but it is not a sufficient one. The creativity threshold required for copyright is minimal, and easily met by a decent API. But just because a work meets this creativity threshold does *not* make it protectable under copyright law.

Copyright is not the only intellectual property law that protects creativity. Inventions that embody creativity are also eligible for protection, but they are the subject matter of patents, not copyrights. As Justice Joseph P. Bradley said in the landmark *Baker v. Selden* ruling in 1879, "To give to the author of the book an exclusive property in the art described therein, when no examination of its novelty has ever been officially made, would be a surprise and a fraud upon the public. That is the province of letters-patent[7], not of copyright."[20 at 102] Similarly, discovering laws of nature or theorems of mathematics requires great creativity, and the resulting laws and theorems can be works of great beauty, but they are eligible for neither copyright nor patent protection.[33]

APIs, like these creations, may well be ineligible for copyright protection under Section 102(b) of the Copyright Act, as Judge Alsup ruled at the first *Oracle v. Google* trial. In his ruling, Alsup took into account that both Sun and Oracle had patents on interface designs.[28 at 996] Sadly, the CAFC disagreed, opining that interfaces were eligible for both patent and copyright protection. The Supreme Court declined to rule on this issue, so we don't have a definitive answer.

2.9 Computer languages are inseparable from their standard libraries.

Like nearly every other programming language, Java comes with a set of standard libraries for performing various tasks such as input/output (I/O) and mathematical operations. Oracle conceded that Google was free to use the Java programming language in developing Android, but claimed infringement of the copyright in the APIs of Java's standard libraries (beyond the 170 API declarations directly mentioned in *The Java Language Specification*). This position is untenable, as a language is largely inseparable from its standard libraries. [8]

As a simple example of the absurdity of Oracle's position, consider the first Java program that a student typically encounters:

```
public class Hello {
  public static void main(String[] args) {
    System.out.println("Hello world!");
  }
}
```

This venerable little program displays "Hello world!" on the console. As strange as it may seem, one cannot write even this trivial program using just the API elements that Oracle claimed were sufficient to implement the language, those specifically mentioned in the normative prose [9] of *The Java Language Specification*, Third Edition.[10][11] These API elements do *not* include `PrintStream`'s `println` method, which is invoked by `System.out.println`.

Simply put, it makes no sense to say that everyone is allowed to reimplement a language if one then prohibits them from reimplementing the APIs of its standard libraries. While these libraries aren't technically a part of the language, they're inseparable from it. If one were to substitute nonstandard APIs for those of the standard libraries, Java programmers would be unable to use the resulting dialect without retraining, and the resulting programs would barely be recognizable as Java programs. Moreover, they would no longer be compatible with existing Java implementations. Clearly, no software professional would consider this a reasonable suggestion. However, this is precisely what Oracle argued that Google should have done, and the CAFC found its argument persuasive.

Kernighan and Ritchie recognized the underlying principle in their 1978 classic, *The C Programming Language*[14:143]:

> The [standard library] routines are meant to be "portable," in the sense that they will exist in compatible form on any system where C exists, and that programs which confine their system interactions to facilities provided by the standard library can be moved from one system to another essentially without change.

Java's designers understood it too. In fact, the first edition of The Java Language Specification[11] contained the API specifications for several packages in the Java standard libraries. They were removed from subsequent editions primarily because they added too much bulk to what became an increasingly large book as the language evolved.

Judge Alsup understood the principle when he said "Contrary to Oracle, there is no bright line between the language and the API."[28 at 982] Justice Breyer understood it when he said:

> [Google] copied the API (which Sun created for use in desktop and laptop computers) only insofar as needed to include tasks that would be useful in smartphone programs. And it did so only insofar as needed to allow programmers to call upon those tasks without discarding *a portion of a familiar programming language* and learning a new one.[51 at 1203]

2.10 It is impossible in practice to implement a language with only the API elements mentioned in its specification.

Throughout the trial, Oracle contended that only 170 declarations from 62 classes (those specifically mentioned in *The Java Language Specification*) were sufficient to implement the language. In its ruling, the CAFC accepted this contention. In fact, the specification mentions at least 64 classes,[4] but that's not the real issue. While

[7] Or *patents*, as we say nowadays. The older term goes back to England, where the monarch would issue letters that were open to the public, granting an office, right or title.

[8] An exception to this rule is the Rust programming language, which was designed to be useable with a tiny, well-defined subset of its libraries.

[9] By "normative prose" we mean the main text of the specification, excluding any example code.

[10] The Java version at issue in the lawsuit was Java SE 5, better known as "Java 5," released in 2004. It is documented in the third edition of *The Java Language Specification*.

Oracle's claim might appear reasonable to a nontechnical audience, it is flawed and results in a massive undercount.

With apologies to John Donne, no class is an island. Class APIs are defined in terms of one another, and cannot exist in isolation. For example, consider the `Runtime` class, which is mentioned in the *Java Language Specification*. Its API contains the following method, which is used to execute a command in a separate process:

```
public Process exec(String cmd, String[] env,
  File dir) throws IOException;
```

In order for this declaration to be meaningful, one needs to implement its return type, `Process`; the types of its parameters, including `File`; and the type of the exception it throws, `IOException`. While these three types are not mentioned in the language specification, one *cannot* implement the `Runtime` class without implementing them too. We call this an *API dependency*. The `Runtime` API is said to *depend on* the `Process`, `File`, and `IOException` APIs (among others).

And that's not all. In order to implement `Process`, `File`, and `IOException`, of course one has to implement all of *their* API dependencies, and so on, ad infinitum, until one stops drawing in any new classes. In other words, one needs to implement the *reflexive transitive closure under API dependency* of the classes mentioned in the language specification.

If you compute this closure on the 64 classes mentioned in the specification, you will find that one must implement 2,381 methods, constructors, and fields, spread across 178 classes in 10 packages in order to implement the language.[11] While it would be tedious to compute this closure by hand, it's easy to write a computer program to compute it using Java's reflection API.[4]

That said, implementing all of these 2,381 methods, constructors, and fields would *still* not provide a usable implementation of the Java programming language, as the resulting dialect would still lack many classes that are integral to the platform, as discussed in the previous section.

2.11 Google did not deliberately make Android incompatible with Java.

Throughout the litigation, Oracle repeatedly claimed that Google had deliberately made Android incompatible with Java. In fact, Google went to great lengths to ensure as much compatibility as possible. Google's engineers started with an open-source implementation called Apache Harmony[1] that itself aimed to be compatible with Sun's Java implementation (which became Oracle's implementation when they acquired Sun). Then Google ran many tests designed for Java, to ensure that they ran equally well on Android. One of the authors (Bloch) participated in this process.

The root of Oracle's claim that Android was deliberately incompatible with Java is the fact that Google did not reimplement the *entire* set of standard libraries in the Java platform. In particular, Google chose not to implement the APIs that made no sense in the context of a mobile platform, such as `Swing`, which is Java's API for mouse-and-keyboard graphical user interfaces (GUIs). Google carefully implemented all of the parts of Java's standard libraries that made sense in the context of an operating system for mobile devices.

As a result of this effort, Android is far and away the most compatible Java-like platform ever written for small devices. It is *far* more compatible with Java than the hodgepodge of platforms that Sun/Oracle marketed under the brand name "Java ME" (the ME stood for micro edition). Many of these so-called Java implementations (which had names like CDC, CLDC, and MIDP) didn't even implement the *language* properly, lacking any support for floating point arithmetic (the primitive types `float` and `double`).[18] In other words, they only provided support for whole numbers such as 1, 2, and 3, not for fractions such as 1.5 and 98.6. Java ME's support for the standard libraries was also thin, lacking even such basics as the collections classes (`Set`, `List`, etc.). Android, by contrast, contained compatible implementations of every Java class that made sense in a mobile device context.

Of course, Sun (and later Oracle) was free to call its mobile platforms Java because it owned the trademark. Because the name Java was trademarked, Google largely avoided its use in the Android documentation. However, programmers understood and took advantage of the compatibility between Android and Java.

A related misconception is that compatibility is an all-or-nothing proposition. Sun, and later Oracle, helped perpetuate this misconception by designating itself the sole arbiter of "Java Compatibility." When Sun/Oracle used the term "compatible implementation," it had a very special meaning with only a tenuous connection to the computer science definition.

True compatibility between implementations means simply that programs written for one will run well on the other. To the extent that such programs run less well on the latter implementation, or require modification in order to run well on it, the degree of compatibility is less. For example, the Compaq Portable was considered to be nearly 100% compatible with the original IBM PC, in that programs written for the IBM PC nearly always ran well on Compaq Portables.

As Sun, and later Oracle, used the term "compatible," it meant the implementation in question had passed a proprietary test suite known as the *JCK*, or *Java compatibility kit*. In order to pass it, one first had to license the JCK from Sun (later Oracle). The terms of these licenses were typically confidential, and typically entailed paying an unspecified sum of money and agreeing to certain *field-of-use restrictions*. For example, such a restriction might prohibit the use of the implementation on devices whose size was less than some agreed-upon minimum.[13] The purpose of such restrictions seems obvious, and it has *nothing* to do with compatibility.

Unfortunately, the CAFC accepted Oracle's contention that "Google designed Android so that it would *not* be compatible with the Java platform, or the JVM."[32 at 1371] The court, moreover, accepted Oracle's claim that the negotiations between Sun and Google over use of the Java API had failed because of "Google's

[11] One of the authors (Bloch) attempted to testify to this state of affairs at the trial, but his testimony was stricken from the record as he was a mere fact witness and not a court-approved expert.

refusal to make the implementation of its programs compatible with the Java virtual machine or interoperable with other Java programs," a position that Oracle claimed was "anathema to [Sun's] 'write once, run anywhere' philosophy."[32 at 1350]

By contrast, Judge Alsup's 2012 decision recognized that Google's reimplementation of the Java API avoided "re-inventing the wheels in the API from scratch, [so that] programmers can call on the tried-and-true pre-packaged programs in the API…This is the whole point of the API."[28 at 982] Google's use of the declarations, with which Java programmers were familiar, would, he concluded, "achieve a degree of interoperability."[28 at 1000]

Although the Supreme Court did not directly address Google's compatibility argument, it recognized the substantial creativity benefits that flowed from Google's reuse of these declarations. The Court also observed that the jury "heard that Google limited its use of the Sun Java API to tasks and specific programming demands related to Android" and "copied the API (which Sun created for use in desktop and laptop computers) … only insofar as needed to allow programmers to call upon those tasks without discarding a portion of a familiar programming language and learning a new one."[51 at 1203] In this way, the Court recognized that Google made Android as compatible as it could, given key differences in the platforms for which Java and Android were developed.

2.12 Many programs run equally well on Android and Java.

Connected to Oracle's claim that Google had designed Android to be incompatible was its contention that there was no evidence that any programs existed that could run on both the Android and Java platforms. The CAFC accepted that claim at face value. It noted that "Google repeatedly cites to the District Court's finding that Google had to copy the packages so that an app written in Java could run on Android." The CAFC found this argument unpersuasive because Google "cites to no evidence in the record that any such app exists and points to no Java apps that either pre-dated or post-dated Android, that could run on the Android platform."[32 at 1371]

This misconception appears to stem from confusion about the terms "program" and "app." If by "program," one means GUI application, then the claim is trivially true, in that Android does not support the mouse-and-keyboard GUIs for which Java was designed. But if the term "program" includes libraries, clients, servers, frameworks, and tools, as computer scientists would likely presume, then the claim is egregiously wrong.

With few exceptions, *every* (non-graphical) library written for Java runs well on Android, and vice-versa. This is not just a curiosity. Hundreds of libraries totaling billions of lines of code are commonly used on both platforms on a daily basis. They include core libraries such as Guava and Apache Commons, cryptography libraries such as Bouncycastle, I/O libraries such as Okio, dependency injection frameworks such as Dagger, concurrency frameworks such as RxJava, logging frameworks such as Timber, analytics and exception tracking frameworks such as Bugsnag, REST adapters such as Retrofit, JSON parsers such as Jackson and Gson, HTTP clients such as OkHttp and Apache HttpClient, HTTP servers such as Jetty, unit testing frameworks such as JUnit, and so on. This list is far from exhaustive, but includes many of the most important third-party libraries for Java.

It is not only third-party libraries that run equally well on Java and Android, but also parts of the platform itself. Some of the standard Java libraries were implemented by parties who were willing to contribute their code under open-source licenses that permitted its use in Java *and* Android. A notable example is Doug Lea's `java.util.concurrent`, but there are many others. A second example is the sorting routines in `java.util` which were reimplemented by one of the authors (Bloch) after he left Sun, and were contributed back to Oracle as well as to Google.

There is even a category of *application programs* that run on both Java and Android: console apps, which are apps that use only basic line or character oriented I/O in lieu of a GUI. The APIs required for this sort of I/O are found in the `java.io` package, which is fully supported by Android. The Hello World program in Section 2.9 is one example of a console app that runs on Google and Android, and the API dependency computation program mentioned in Section 2.10 is another.

You may wonder how you'd run a console app on a smartphone, which lacks a keyboard. There is a tool called the Android Debug Bridge (adb), which is used with a USB cable and a host system to provide a remote console. While typical end users don't run console apps on Android, developers do so as a matter of course.

Not only are there programs that run *on* both Android and Java, but there are many tools that work equally well with code written *for* both platforms. For example, the build systems Gradle and Maven both work equally well for Android and Java projects, as do the IDEs IntelliJ IDEA and Eclipse.

Simply put, the Java and Android platforms largely comprise a single ecosystem. The great bulk of libraries and tools that support one platform also support the other. Programmers trained on either platform can easily apply their skills to the other.

2.13 Google did not use the Java declarations in Android because they were "popular" with Oracle's "audience."

In an effort to bolster its copyright infringement argument and persuade courts that software was as protectable as more conventional copyrighted works, Oracle's briefs, both to the CAFC and to the Supreme Court, repeatedly characterized the Java API as "popular" and referred to Java programmers as Oracle's "audience" and "fan base."[29,36,39,41,47,50] For example, Oracle's merits brief to the Supreme Court claimed that Google had "committed an egregious act of plagiarism" in a bid to appeal to "Oracle's loyal audience … who had come to know and love Java SE." [50]

In the same brief, Oracle argued that it was "anathema to copyright law … to 'treat the current popularity of [Oracle's] work among developers as retroactively divesting the work of copyright protection.'" A work should not lose copyright, Oracle contended, "just because the audience has expended effort to learn it." It added that "[w]hat an audience has learned, or grown to love, has no bearing on whether the work is protected."[50:32–33]

The fundamental problem with these emotional appeals is that they ignore the functional and compositional nature of software.

Programmers are not "audiences" for the Java declarations, and those declarations are not "popular" with programmers in the way hip-hop songs are popular with fans. These declarations are the fundamental building blocks for constructing new programs, and programmers are artisans who have learned to work with those building blocks.

The Supreme Court had a very different conception of the interests of Java programmers in being able to use familiar APIs to write programs for the Android or Java platforms. The Court regarded Google's use of the declarations as enabling programmers to create new products. "To the extent that Google used parts of the Sun Java API to create a new platform that could be readily used by programmers, its use was consistent with that creative 'progress' that is the basic constitutional objective of copyright itself."[51 at 1203]

The Court observed that "given programmers' investment in learning the Sun Java API, to allow enforcement of Oracle's copyright would risk harm to the public." To allow Oracle to enforce copyright in the API would make the declarations "a lock limiting the future creativity of new programs" and "Oracle alone would hold the key." That lock "would interfere with, not further, copyright's basic creativity objectives" because it would impede programmers' ability to create new programs for the Android platform.[51 at 1208]

2.14 Independent API reimplementation is the rule, not the exception.

Since the earliest days of the computer era, programmers have been reimplementing each other's APIs, often without permission from the API's creators. Examples include the Fortran libraries, the IBM 370 instruction set, the UNIX system calls, the C libraries, the Hayes "AT" modem command set, the DEC VT-100 terminal control codes, the IBM PC BIOS, the Win32 APIs, and the Amazon Simple Storage Service (S3). This list is not intended to be exhaustive, but to convey the general flavor of historical reimplementations.

In all of these cases, one party developed an API and successfully marketed a product that implemented it. Then other parties launched products that implemented the same API so that customers who had acquired skills, software, or hardware for the first product could make use of them on the later products. Often the API *far* outlasted the platform for which it was written. For example, the Hayes modem command set is still used in modern cell phones (for communication between the main processor and the baseband processor), though telephone modems are long gone. Similarly, the DEC VT-100 terminal control codes are implemented by the terminal emulators that ship with every major desktop operating system, though computer terminals are a thing of the past.

Much of this reimplementation has led to thriving, competitive industries, such as IBM-compatible mainframes and peripherals, telephone modems, computer terminals, IBM-compatible PCs, software and hardware for IBM compatible PCs, and cloud-computing services. The resulting software and hardware ecosystems were highly interoperable. It was possible to buy components from different manufacturers and reasonably expect them to work together. If a component was insufficiently compatible, users and potential users would quickly become aware of the problem, after which the problem would either be fixed or the insufficiently compatible component would fail in the marketplace.

It is also worth noting that Sun (now Oracle America) reimplemented existing APIs as part of the Java platform itself. For example, Java reimplemented C's math API, enabling C programmers to transition to Java with ease. Along similar lines, one of the authors (Bloch) oversaw Sun's reimplementation of the Perl programming language's regular expression API in the Java platform. In perhaps the ultimate irony, Oracle went on to accuse Google of reimplementing the Java package that encapsulates Perl's regular expression API: `java.util.regex` was one of the 37 packages at issue in Oracle's lawsuit.

Finally, it is worth noting that both Sun and Oracle were members of the American Committee for Interoperable Systems. This committee filed briefs in several prominent software copyright cases and wrote a letter to the Clinton administration arguing that interfaces necessary for compatibility were unprotectable by copyright law.[7]

For a historical example of API reimplementation, see Unix co-creator Dennis Ritchie's `alt.folklore.computers` post from April 10, 1998.[17] In this post, Ritchie described an early 1980s visit to the Mark Williams Company (MWC), which had written an early Unix-like operating system called Coherent. The system reimplemented most of Unix, including its system calls, standard libraries, and command line interface. Ritchie visited MWC with an AT&T lawyer in tow to look for signs of foul play. Ritchie recalled: "...looking at various corners convinced me that I couldn't find anything that was copied … If it came to it, I could never honestly testify that … what they generated was irreproducible from the manual." And he concluded: "'we' (=AT&T) backed off, possibly after other thinking and investigation that I wasn't involved in. So far as I know, after that MWC and Coherent were free to offer their system and allow it to succeed or fail in the market."

These examples illustrate that, far from being an outrageous act of misappropriation, API reimplementation is a normal, expected stage in the lifecycle of the API. If an API is successful, it is likely that it will eventually be reimplemented, perhaps repeatedly, and that the API will outlive the platform for which it was created. The industry has long recognized that reimplementation of interfaces is pro-innovation and non-infringing. Courts have sanctioned this practice in several high-profile cases.[24–27]

3. CONCLUDING THOUGHTS

Both authors of this paper submitted briefs to the Supreme Court urging it to rule that the declarations at issue in *Google v. Oracle* were unprotectable by copyright law.[48,49] While the Court decided not to rule on the copyrightability issue, its ruling that Google's reuse of the declarations was fair use as a matter of law sent a strong signal that programmers should be able to reimplement program interfaces to achieve a degree of interoperability free from copyright liability.

If the Court had ruled that APIs were unprotectable by copyright law, this would have guaranteed that programmers could

reimplement them without any copyright infringement litigation risk. Instead, because courts decide fair use defenses on a case-by-case basis, some litigation risk remains. Yet the Supreme Court's decision that Google's reimplementation of the Java declarations was fair use *as a matter of law* sets a strong precedent for other courts to follow.

The Court considered the following factors as supporting Google's fair use defense:

- The functional nature of the declarations
- The manner in which they were "inextricably bound" up with method calls and implementing code
- The interests of programmers in being able to make use of their experiences and investments in learning Java
- The creativity that reuse of the declarations had unleashed
- The transformative character of Android as a smartphone platform, as compared with the desktops and laptops for which Sun developed Java
- The public benefit that copying declarations will likely produce

The Court also recognized that the jury had heard testimony that reimplementing interfaces was common in the computing field. Hopefully, the Court's ruling will be sufficient to preserve the freedom to reimplement interfaces that programmers have enjoyed since the dawn of the computer era, a practice that enabled the astronomical growth that the computing field has experienced in that time.

There are ongoing litigations that will test whether emulating another program's functionality, having the same input and output formats as another program, or enabling virtualization of software to test its security are copyright infringements.[51,58] While the Supreme Court's *Google* decision should be helpful in these and many future software copyright cases, the struggle over copyright protection for interfaces is far from over.

ACKNOWLEDGMENTS

We thank Clark Asay, Michael Barclay, Dan Bloch, Kevin Bourrillion, William Chargin, Bob Glushko, James Grimmelmann, Kathryn A Hashimoto, Phil Malone, Jef Pearlman, Tim Peierls, Alfred Spector, Peter Stout, and anonymous reviewers for their careful reading of earlier drafts of this paper, and for their many helpful comments. We thank Cindy Bloch for the hard work that she put into the bibliography and copyediting.

REFERENCES

[1] Apache Software Foundation. *Apache Harmony - Open Source Java SE.* Retrieved from https://harmony.apache.org/

[2] Joshua Bloch. 2006. How to design a good API and why it matters. In *Companion to the 21st ACM SIGPLAN conference on Object-oriented programming systems, languages, and applications - OOPSLA '06*, ACM Press, New York, New York, USA. DOI:https://doi.org/10.1145/1176617.1176622

[3] Joshua Bloch. 2018. *Effective Java.* Addison Wesley Professional, Boston, MA.

[4] Joshua Bloch. 2022. *Java 5 API Dependencies.* Github. Retrieved March 9, 2022 from https://github.com/jbloch/Java-5-API-Dependencies

[5] Joshua Bloch and Neal Gafter. 2005. *Java Puzzlers: Traps, Pitfalls, and Corner Cases.* Addison Wesley Professional, Boston, MA.

[6] Stephen Breyer. 1970. The Uneasy Case for Copyright: A Study of Copyright in Books, Photocopies, and Computer Programs. *Harv. Law Rev.* 84, 2 (1970), 281–351. DOI:https://doi.org/10.2307/1339714

[7] Peter M. C. Choy. 1992. Retrieved from https://www.ccianet.org/wp-content/uploads/2014/10/ACIS-Letter-to-Clinton-Admin-1992.pdf

[8] Ira W. Cotton and Frank S. Greatorex Jr. 1968. Data Structures and Techniques for Remote Computer Graphics. In *Managing Requirements Knowledge, International Workshop on*, ACM, 553–544. DOI:https://doi.org/10.1109/AFIPS.1968.82

[9] Brian Goetz, Tim Peierls, Joshua Bloch, Joseph Bowbeer, David Holmes, and Doug Lea. 2006. *Java Concurrency in Practice.* Addison Wesley Professional, Boston, MA.

[10] Thomas C. Goldstein. Attorney & Partner at Goldstein & Russell. January 24 2022.

[11] James Gosling, Bill Joy, Guy L. Steele Jr, and Gilad Bracha. 2005. *The Java Language Specification* (3rd ed.). Addison Wesley, Boston, MA.

[12] Sarah Jeong. 2017. How the judge on Oracle v. Google taught himself to code. *The Verge.* Retrieved March 3, 2022 from https://www.theverge.com/2017/10/19/16503076/oracle-vs-google-judge-william-alsup-interview-waymo-uber

[13] James Kao. 2007. Apache Harmony Questions Sun Regarding JCK License Terms. *InfoQ.* Retrieved March 1, 2022 from https://www.infoq.com/news/2007/04/openletter/

[14] Brian W. Kernighan and Dennis M. Ritchie. 1978. *The C Programming Language.* Prentiss Hall, Englewood Cliffs, NJ.

[15] Mark A. Lemley and Pamela Samuelson. 2021. Interfaces and Interoperability After Google v. Oracle. *Tex. Law Rev.* 100, (2021), 1. Retrieved from https://texaslawreview.org/wp-content/uploads/2021/11/1Lemley.Printer.pdf

[16] J. F. Ossanna, L. E. Mikus, and S. D. Dunten. 1965. Communications and Input/Output Switching in a Multiplex Computing System. In *Proceedings of the November 30-December 1, 1965, Fall Joint Computer Conference, Part I*, ACM Press, New York, New York, USA, 231–241. DOI:https://doi.org/10.1145/1463891.1463916

[17] Dennis Ritchie. 1998. Coherent. *alt.folklore.computers.* Retrieved March 2, 2022 from https://groups.google.com/g/alt.folklore.computers/c/_ZaYeY46eb4/m/5B41Uym6d4QJ?pli=1

[18] Sun Microsystems. 2000. *Connected, Limited Device Configuration, Specification Version 1.0a, Java 2 Platform Micro Edition.*

[19] *Copyright Act of 1976.* Retrieved from https://www.copyright.gov/title17/title17.pdf

[20] 1879. *Baker v. Selden.* Retrieved from https://casetext.com/case/baker-v-selden

[21] 1980. *An Act to Amend the Patent and Trademark Laws.* Retrieved August 11, 2022 from https://www.govinfo.gov/content/pkg/STATUTE-94/pdf/STATUTE-94-Pg3015.pdf

[22] 1983. *Apple Computer, Inc. v. Formula Intern. Inc.* Retrieved from https://casetext.com/case/apple-computer-inc-v-formula-intern-inc-3

[23] 1986. *Whelan Associates, Inc. v. Jaslow Dental Laboratory, Inc.* Retrieved from https://scholar.google.com/scholar_case?case=10382786109829050440

[24] 1992. *Sega Enterprises Ltd. v. Accolade, Inc.* Retrieved from https://scholar.google.com/scholar_case?case=12221231553971530035

[25] 1992. *Computer Associates Intern., Inc. v. Altai, Inc.* Retrieved from https://scholar.google.com/scholar_case?case=6976925648486076739

[26] 1995. *Lotus Development Corp. V. Borland intern, 49 F.3d 807.* Retrieved March 14, 2022 from https://casetext.com/case/lotus-development-corp-v-borland-intern

[27] 2000. *Sony Computer Entertainment v. Connectix Corp.* Retrieved from https://scholar.google.com/scholar_case?case=7166769136737271634

[28] 2012. *Oracle Am., Inc. v. Google Inc.* Retrieved from https://casetext.com/case/oracle-am-inc-v-google-inc-3

[29] 2013. Oracle v. Google Opening Brief and Addendum of Plaintiff Appellant. Retrieved March 9, 2022 from https://s3.amazonaws.com/cdn.orrick.com/files/Oracle-Brief.PDF

[30] 2013. Oracle v. Google Brief of Amici Curiae Intellectual Property Professors in Support of Defendant-Cross Appellant and Affirmance.

[31] 2013. Oracle v. Google Response and Reply Brief of Plaintiff-Appellant. Retrieved August 14, 2022 from http://www.groklaw.net/pdf4/OraGoogleAppeal-134.pdf

[32] 2014. *Oracle America, Inc. v. Google Inc.* Retrieved from https://scholar.google.com/scholar_case?case=15197092051369647665

[33] 2014. *Alice Corp. v. CLS Bank International.* Retrieved from https://casetext.com/case/alice-corp-v-cls-bank-intl

[34] 2014. Google v. Oracle Brief of Amici Curiae Computer Scientists in Support of Petitioner. Retrieved from https://www.eff.org/files/2014/11/07/google_v_oracle_computer-scientists-certpetition-amicus-brief_14-410_final.pdf

[35] 2014. Google v. Oracle Brief of Amici Curiae Intellectual Property Professors in Support of Grant of Petition. Retrieved from https://www.law.berkeley.edu/files/Google_v_Oracle_cert_pet_IP_Profs_amicus.pdf

[36] 2014. Google v. Oracle Brief in Opposition. Retrieved March 9, 2022 from https://www.eff.org/files/2014/12/09/oracle_brief_in_opposition_to_petition_for_writ_of_certiori.pdf

[37] 2015. Google Inc. V. Oracle America, Inc. *SCOTUSblog*. Retrieved March 15, 2022 from https://www.scotusblog.com/case-files/cases/google-inc-v-oracle-america-inc/

[38] 2016. *Oracle Am., Inc. v. Google Inc.* Retrieved from https://casetext.com/case/oracle-am-inc-v-google-inc-20

[39] 2017. Oracle v. Google Opening Brief and Addendum for Oracle America, Inc. Retrieved March 9, 2022 from https://patentlyo.com/media/2017/02/02-10-17_Oracle-opening-brief-second-appeal.pdf

[40] 2017. Oracle v. Google Brief of Computer Scientists as Amici Curiae in Support of Defendant Appellee. Retrieved from https://www-cdn.law.stanford.edu/wp-content/uploads/2016/03/Oracle-v.-Google-Brief-of-Amici-Curiae-Computer-Scientists-FINAL-1.pdf

[41] 2017. Oracle v. Google Corrected Response and Reply Brief for Oracle America, Inc. Retrieved March 9, 2022 from https://www.eff.org/files/2017/08/15/2017.08.04_oracle-corrected-response-and-reply-brief.pdf

[42] 2018. *Oracle Am., Inc. v. Google LLC ., no.* Retrieved March 14, 2022 from https://law.justia.com/cases/federal/appellate-courts/cafc/17-1118/17-1118-2018-03-27.html

[43] 2018. Oracle v. Google Brief of Computer Scientists as Amici Curiae in Support of Petition for Rehearing En Banc. Retrieved from https://www.eff.org/files/2018/06/13/oracle_v_google_computer-scientists-en-banc-amicus-brief.pdf

[44] 2018. Oracle v. Google Brief for Copyright Law Professors Pamela Samuelson and Clark Asay as Amici Curiae in Support of Defendant-Cross Appellant Google LLC's Petition for Rehearing En Banc. Retrieved from https://www.eff.org/files/2018/06/13/17-1118_-1202_brief_for_copyright_law_professors_pamela_samuelson_and_clark_asay_as_amici_curia.pdf

[45] 2019. Google v. Oracle Brief of 78 Amici Curiae Computer Scientists in Support of Petitioner. Retrieved from https://www.supremecourt.gov/DocketPDF/18/18-956/89487/20190225134131839_18-956_Oracle_v__Google_Computer_Scientists_Amicus_Motion_Brief_FILE.pdf

[46] 2019. Google v. Oracle Brief of 65 Intellectual Property Scholars as Amici Curiae in Support of Petitioner. Retrieved from https://www.supremecourt.gov/DocketPDF/18/18-956/89474/20190225131314910_IP%20Scholars%20Amicus%20Brief.pdf

[47] 2019. Google v. Oracle Brief in Opposition. Retrieved March 9, 2022 from https://www.supremecourt.gov/DocketPDF/18/18-956/93436/20190327160337558_190311%20for%20E-Filing.pdf

[48] 2020. Google v. Oracle Brief Amici Curiae of 83 Computer Scientists in Support of Petitioner. Retrieved from https://www.supremecourt.gov/DocketPDF/18/18-956/128391/20200113145027664_18-956%20Google%20v%20Oracle%20Computer%20Scientists%20Merits%20Amicus%20FOR%20FILING.pdf

[49] 2020. Google v. Oracle Brief of 72 Intellectual Property Scholars as Amici Curiae in Support of Petitioner. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3518887

[50] 2020. Google v. Oracle Brief for Respondent. Retrieved March 9, 2022 from https://www.supremecourt.gov/DocketPDF/18/18-956/132891/20200212180251262_200208a%20Resp%20Brief%20for%20efiling.pdf

[51] 2021. *Google LLC v. Oracle America, Inc.* Retrieved from https://scholar.google.com/scholar_case?case=4050620474768552042

[52] 2021. *Google LLC v. Oracle America, Inc.* Retrieved from https://www.supremecourt.gov/opinions/20pdf/18-956_d18f.pdf

[53] 2021. SAS Institute Inc. V. World Programming Ltd. *Fed Circuit Blog*. Retrieved March 15, 2022 from https://fedcircuitblog.com/other-cases/sas-institute-inc-v-world-programming-limited/

[54] 2021. *The David Rubenstein Show: Peer to Peer Conversations*. Bloomberg, USA. Retrieved March 3, 2022 from https://archive.org/details/BLOOMBERG_20211219_190000_The_David_Rubenstein_Show_Peer_to_Peer_Conversations/start/360/end/420

[55] Google LLC v. Oracle America Inc. *SCOTUSblog: Independent News and Analysis on the U.S. Supreme Court*. Retrieved March 1, 2022 from https://www.scotusblog.com/case-files/cases/google-llc-v-oracle-america-inc/

[56] Google Books NGram Viewer. *Google Books*. Retrieved March 1, 2022 from https://books.google.com/ngrams

[57] Google Books Ngram Viewer ("software interface, declaring code, application programming interface"). *Google Books*. Retrieved March 1, 2022 from https://books.google.com/ngrams/graph?content=software+interface%2Cdeclaring+code%2Capplication+programming+interface&year_start=1950&year_end=2019&corpus=26&smoothing=3

[58] Google Scholar ("declaring code"). *Google Scholar*. Retrieved March 1, 2022 from https://scholar.google.com/scholar?hl=en&as_sdt=0%2C5&q=%22declaring+code%22&btnG=

[59] *Apple Inc. v. Corellium LLC*. Retrieved from https://www.pacermonitor.com/public/case/41494334/Apple,_Inc_v_Corellium,_LLC